



endingo

정훈이의 공부일지

CATEGORY



👋 안녕하세요, AI 개발자 이정훈입니다

AI · 백엔드 · 데이터 분석 중심의 프로젝트와 경험을 공유합니다.

Contact Me

Data science/Agent

[Agent] 7. Agent 시스템: RAG

endingo 2025. 5. 4. 13:00 ⋮

1. RAG를 LangGraph로 구현하기

- 간단한 RAG
- Vector DB를 tool로 사용

1) 간단한 RAG

- State 정의

```
class State(TypedDict):  
    messages: Annotated[list, add_messages]
```

- Vector DB 정의
- 단순 프롬프트만 응답형태

```
from langchain.document_loaders import CSVLoader
```

```
# CSV 파일 로드  
file_name = "sample.csv"  
csv_loader = CSVLoader(file_path= path + file_name)
```



정훈이의 공부일지



```
embedding = OpenAIEmbeddings(model="")
vectorstore = Chroma.from_documents(documents_csv, embedding,
                                     persist_directory= path + "")
```

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

- Node 정의

최근 사용자의 질문을 꺼내서 RAG 체인으로 답변을 얻고 새로운 GPT 응답 메시지를 state["messages"]에 추가하는 역할

메시지 리스트 중에서 HumanMessage만 골라내서 가장 마지막 메시지를 선택하여 m.content로 텍스트만 추출

```
llm = ChatOpenAI(model="")
rag_chain = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)
```

```
def rag_node(state: State):
    user_msg = [m.content for m in state["messages"] if isinstance(m, HumanMessage)][-1]
    answer = rag_chain.invoke({"query": user_msg})
    return {"messages": [AIMessage(content=answer["result"])]}
```

2) VectorDB를 Tool로 사용

GPT가 스스로 판단해서 검색할지 말지를 결정: Reasoning + Acting

단순히 프롬프트 응답형태가 아니라 질문에 답하기 위해 외부 툴을 쓸지 말지를 LLM이 스스로 판단

- Vector DB 준비

여기까지는 위에랑 똑같음

```
from langchain.document_loaders import CSVLoader
```

CSV 파일 로드

```
csv_path = "sample.csv"
csv_loader = CSVLoader(file_path= path + csv_path)
documents_csv = csv_loader.load()
```

벡터 DB 정의

```
embedding = OpenAIEmbeddings(model="")
vectorstore = Chroma.from_documents(documents_csv, embedding,
                                     persist_directory= path + "")
```

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
```

- Tool 정의

상위 3개의 문서를 벡터스토어에서 검색하고 검색된 문서 내용들을 줄바꿈으로 구분하여 하나의 문자열로 반환
그리고 툴리스트에 vectordb_search에 집어넣고 툴실행 노드에 툴을 선언



정훈이의 공부일지



```
docs = retriever.get_relevant_documents(query)
```

```
return "\n\n".join(d.page_content for d in docs)
```

```
tools = [vectordb_search]
```

```
tool_node = ToolNode(tools)
```

- llm에 Tool 바인딩

```
llm = ChatOpenAI(model="", temperature=0).bind_tools(tools)
```

- 노드 준비 & 분기함수

마지막 노드에 톨 콜스 내용이 포함되면 tool_calls로 이동 아니면 END

```
def call_model(state: State):
    messages = state["messages"]
    response = llm.invoke(messages)
    return {"messages": [response]}
```

```
def should_continue(state: State) -> Literal["tools", END]:
    last = state["messages"][-1]
    if getattr(last, "tool_calls", None):
        return "tools"
    return END
```

- 그래프 구성

```
builder = StateGraph(State)
builder.add_node("call_model", call_model)
builder.add_node("tools", tool_node)
```

```
# connect nodes
builder.add_edge(START, "call_model")
builder.add_conditional_edges("call_model", should_continue)
builder.add_edge("tools", "call_model")
```

```
graph = builder.compile()
```

공감



정훈이의 공부일지



[Agent] 2. Agent 시스템 State, Node, Edge 기... (0)

정훈이의 공부일지

개발하면서 겪는 시행착오와 배움을 기록합니다. 백엔드, AI, 데이터 분석, 프로젝트 경험 중심으로 운영 중입니다. 공부하면서 얻은 인사이트를 함께 나누고 싶어요!

댓글 0

endingo

내용을 입력하세요.

등록

공지사항

최근에 올라온 글

- [Agent] 7. Agent 시스템: RAG
- [Agent] 6. Agent 시스템: Me...
- [Langchain] 5. 메모리와 체인
- [Langchain] 4. Prompt와 Ou...

최근에 달린 댓글

- endingo님 솔직하고 담백한 표...
- 잘 읽고 갑니다
- 오늘도 화이팅!!!
- 덕분에 좋은 정보 얻어 갑니다 ! ...

Total

317

Today	0
Yesterday	0

링크

TAG

more

- github
- git

« 2025/05 »						
일	월	화	수	목	금	토
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

글 보관함

- 2025/05 (8)
- 2025/04 (34)
- 2025/03 (7)



정훈이의 공부일지



AI와 엑셀, 데이터 분석을 좋아하는 개발자입니다.
다양한 문제 해결과 창의적인 개발을 추구합니다.
 이메일 보내기